Chapter 4: Developing Requirements

Definitions

Domain, problem, and scope

- **Domain:** The business or technical field in which the software operates.
- **Domain analysis:** Learning about the domain, its concepts, procedures, and problems before and during development.
- **Domain expert:** A person with deep knowledge of the field in which the software will operate.
- **Domain document:** A reference describing domain knowledge, terminology, stakeholders, environment, existing tasks, and related systems.
- **Glossary:** A collection of terms and their agreed meanings within a domain or project.
- **Stakeholder:** A person or organization affected by or interested in a system and its development.
- **Customer:** The person or organization requesting or purchasing the system.
- **User:** A person who interacts with the system to accomplish tasks.
- **Problem statement:** A concise description of the customer's difficulty or opportunity before selecting a solution.
- **Scope:** The boundary identifying which capabilities and responsibilities belong inside or outside the proposed work.
- **Greenfield project:** A project that develops a new system rather than evolving an existing one.

Requirements and quality measures

- **Requirement:** A statement specifying what a proposed system must do or a constraint on the system or its development.
- **Requirements specification:** The recorded set of agreed system requirements and constraints.
- **Functional requirement:** A requirement describing system behaviour, including inputs, outputs, stored data, computations, and ordering or synchronization of actions.
- **Quality requirement:** A measurable requirement specifying how well the system performs, such as response time, reliability, or resource use.
- **Platform requirement:** A constraint on the technology or environment in which the system must operate.

- **Process requirement:** A constraint on development activities, methods, deliverables, or the project plan.
- **Constraint:** A restriction that a system or its development must satisfy.
- **Response time:** The elapsed time between a defined request or event and the required response, measured under stated conditions.
- **Throughput:** The amount of work completed per unit of time under stated conditions.
- **Resource use:** The amount of processor time, memory, storage, bandwidth, or other resources consumed.
- **Reliability:** The ability to perform required functions without failure under stated conditions for a specified period.
- **Availability:** The extent to which a system is operational and accessible when needed.
- **Recovery:** Restoring service and an acceptable state after a failure.
- **Maintainability:** The ease of understanding, correcting, and modifying software.
- **Reusability:** The ability to use software components in other systems or contexts.
- **Synchronization:** Coordination of activities so that their relative timing or order satisfies required rules.
- **Workload:** The volume and pattern of requests or activity under which system behaviour is measured.

Use cases and scenarios

- **Use case:** A description of interactions through which an actor achieves a goal with the system.
- **Actor:** A role played by a user or an external participant, including another system, that interacts with the system being modelled.
- **Goal:** The outcome an actor seeks to achieve through interaction with the system.
- **Use-case model:** A collection of use cases and descriptions or diagrams of their relationships.
- **Precondition:** A condition that must already hold before a use case or operation begins.
- **Postcondition:** A condition describing the state that must hold after a use case or operation finishes.
- **Basic flow:** The main sequence of actor actions and system responses for accomplishing the use-case goal.
- **Alternative flow:** A valid variation from the main interaction sequence.

- **Exceptional flow:** A sequence handling an error or another condition that prevents normal progress.
- **Include relationship:** A relationship that factors required shared behaviour into a separate use case; its dependency points from the including case to the included case.
- **Extend relationship:** A relationship adding conditional or optional behaviour to a base use case at an extension point; its dependency points from the extending case to the base case.
- **Extension point:** A named location in a base use case where extending behaviour may be inserted.
- **Generalization:** A relationship in which a specialized use case or actor inherits common characteristics from a more general one.
- **Scenario:** One concrete occurrence of a use case with particular actors, data, and events.
- **User story:** A short statement of a stakeholder need, refined through discussion and acceptance criteria.
- **Acceptance criteria:** Specific conditions used to determine whether a requirement or user story has been satisfied.
- **Prioritization:** Ordering requirements or use cases by considerations such as value, risk, importance, and dependencies.

Gathering, reviewing, and managing requirements

- **Requirements gathering / elicitation:** Discovering needs and constraints from stakeholders, existing documents, work practices, and other sources.
- **Requirements analysis:** Examining and organizing gathered information to resolve conflicts, clarify meaning, and establish coherent requirements.
- **Observation:** Learning about requirements by watching users perform their work and examining the context and exceptions.
- **Interview:** A discussion using questions to learn about stakeholder needs, procedures, and goals.
- **Brainstorming:** A facilitated group activity for generating ideas around a defined question or problem.
- **Joint Application Development (JAD):** Intensive collaborative sessions in which stakeholders and developers work together to define requirements and proposed solutions.
- **Prototype:** A preliminary representation or implementation used to explore ideas and obtain feedback; it may omit production functionality.
- **UI mock-up:** A representation of proposed screens or interactions used to discuss the user interface.

- **Use-case analysis:** Identifying actors, their goals, and the interactions needed to accomplish those goals.
- **Requirements validation:** Reviewing whether requirements accurately express stakeholder needs and describe a useful, achievable system.
- **Unambiguous requirement:** A requirement with one clear intended interpretation.
- **Consistent requirements:** Requirements that do not contradict one another.
- **Feasible requirement:** A requirement achievable within the available technology, resources, and constraints.
- **Verifiable requirement:** A requirement whose satisfaction can be assessed using an objective test, inspection, analysis, or demonstration.
- **Completeness:** Adequate coverage of required behaviour, conditions, and constraints within the agreed scope.
- **Traceability:** Links connecting requirements to their sources, related requirements, design, implementation, and tests.
- **Change-impact analysis:** Assessing how a proposed change affects requirements, design, implementation, tests, effort, and schedule.
- **Requirements management:** Maintaining requirements and their traceability while evaluating and controlling changes over time.
- **Correction:** A change that resolves an error in an existing requirement or implementation.
- **Enhancement:** A change that adds or expands capabilities, potentially increasing scope.

4.0 Domain Analysis

Domain analysis is learning about the business or technical field where software will operate. A domain expert has deep knowledge of that field.

- Understanding the domain speeds development, improves the solution, and helps anticipate extensions.
- A domain document includes a summary, glossary, general knowledge, customers and users, environment, current tasks and procedures, competing software, and similarities to other domains.
- Keep it accessible and maintainable, with useful diagrams and links.

4.1 Starting Points, Problem, and Scope

- Projects can start with a new opportunity or a problem in an existing system. Greenfield projects develop new systems.
- State the customer's difficulty or opportunity briefly before choosing a solution.
- Scope defines the work's boundary. List potential capabilities, then decide what belongs inside and outside it.
- Narrow an overly broad problem; revisit higher-level goals if the scope is too narrow.

4.2 What is a Requirement?

A requirement specifies what the proposed system must do or a constraint on its development. Together, requirements form a negotiated agreement among stakeholders.

4.2.0 Writing a good requirement

- Identify the system, required action or outcome, relevant conditions, and a measurable success criterion.
- Use complete sentences and consistent terminology. Use shall for mandatory behaviour and make optional status explicit.
- Specify the outcome without unnecessarily dictating implementation. Confirming a purchase need not require Microsoft Outlook unless that is a genuine platform constraint.
- Slide example: the online banking system shall allow an Internet user to access her current account balance in less than five seconds.
- For testing, clarify the response-time measurement's start, end, and workload conditions.

4.2.1 Pitfalls

- Avoid vague terms such as quickly, user-friendly, flexible, minimal, or usually unless measurements define them.
- Avoid speculative wishes, incomplete sentences, and assumptions about what users probably want.
- Avoid impossible promises such as handling every conceivable failure or running on every platform.
- State obligations for the system. Describing what a user happens to do does not adequately specify system behaviour.

4.3 Types of Requirements

- Functional requirements describe inputs, outputs, stored data, computations, and required timing or synchronization of actions.
- Quality requirements specify measurable response time, throughput, resource use, reliability, availability, recovery, maintainability, or reusability.
- Platform requirements constrain the environment and technology.
- Process requirements constrain development activities, methods, or the project plan.
- Retrieving a balance is functional behaviour; the response-time limit is a related quality requirement.

4.4 Use Cases

A use case describes interactions through which an actor achieves a goal with the system. Actors are roles played by users or external participants.

- Focus on observable interactions, avoiding internal algorithms and unnecessary screen-design details.
- A use-case model contains use cases and optionally a description or diagram of their relationships.

4.4.0 Description format

- Name: short task description. Actors: participating roles. Goals: what those roles seek to achieve.
- Preconditions: what must already be true. Summary: brief overview. Related use cases: dependencies or specializations.

- Steps: actor actions and system responses, commonly in two columns. Include alternative and exceptional paths.
- Postconditions: the state after completion. Distinguish success and failure where necessary.

4.4.1 Relationships

- Include factors shared behaviour into another use case. The dependency points from the including case to the included case.
- Extend adds conditional or optional behaviour at an extension point. The dependency points from the extending case to the base case.
- Generalization captures common behaviour with specialized variants. The arrow points toward the more general case or actor.
- Inclusion reuses a lower-level task; extension keeps conditional behaviour separate from the basic flow.

4.4.2 Scenarios and priorities

- A scenario is a concrete occurrence of a use case with specific actors, data, and events.
- Prioritize central use cases, risky functions, and work with high commercial or stakeholder value.
- Use cases help define scope, plan development, validate requirements, derive tests, and organize user manuals.
- Agile user stories are typically shorter and less formal statements of needs. Discussion and acceptance criteria supply implementation detail.

4.4.3 Study example: registering for a course

- Actor: student. Goal: obtain a course place. Preconditions: authenticated student and open registration period.
- The student requests a course. The system displays details and availability. The student requests enrolment. The system checks eligibility and capacity, records the enrolment, and confirms it.
- If capacity or prerequisites prevent enrolment, the system explains the rejection and does not record successful enrolment.
- Success postcondition: an enrolment links the student to the course offering. This is an explanatory study example.

4.5 Gathering and Analysing Requirements

- Observation: review documents and watch users work; ask them to explain decisions and exceptions.
- Interviews: ask about details, future goals, alternatives, other information sources, and helpful diagrams.
- Brainstorming: use a moderator and a clear trigger question. Joint Application Development uses intensive collaborative sessions.
- Prototypes: paper sketches or UI mock-ups help users react to proposed interactions; they may omit computation, databases, and integration.
- Use-case analysis identifies actors and the tasks each needs to accomplish.
- Repeat these techniques as understanding improves. More detail is needed for complex systems, external interfaces, technical domains, and costly consequences of error.

4.6 Reviewing Requirements

- Each requirement should be valuable, relevant, clear, unambiguous, consistent, feasible, verifiable, and uniquely identifiable.
- Avoid unnecessary design restrictions and check that the resulting system can meet quality goals.
- Review the entire set for completeness, organization, consistency, and stakeholder agreement.
- Traceability links requirements to their origins, design, implementation, and tests, supporting change-impact analysis.

4.6.0 Ambiguity example

The slide's interest-calculation example asks for edits to one of three fields to compute the other two. One value generally cannot determine two unknowns. Specify which additional value remains fixed and how each edit is interpreted.

4.7 Managing Changing Requirements

- Requirements change with business processes, technology, and improved understanding. Analysis continues throughout development.
- Assess each change's benefit, effort, design effects, schedule impact, and affected tests.
- Keep clients and users involved. Small presentation changes may be inexpensive; structural changes can require significant redesign.

- Distinguish corrections from enhancements that expand scope. Update specifications and traceability after accepting a change.